

Research Project: Implementation of an In-Memory Key-Value Store Based on Optimized B+ Trees

Janne Gruhler, Maxim Bellm, Albert Rico Martin, Matthias Raba

Department of Computer Science

Hochschule Furtwangen

Furtwangen, Germany

Abstract—B⁺-trees have long been considered inferior to hash-based structures for in-memory workloads. Recent work on optimized node layouts argues that this gap is largely an artifact of the implementation rather than of the data structure itself. We used the authors’ reference implementation and converted it into a reusable static library and attempted to reproduce their measurements. This succeeded only in part: the datasets are described but not distributed, benchmarks against self-generated substitutes crashed, and even after obtaining the original datasets our absolute throughput still deviates by orders of magnitude, though most qualitative trends hold. On this basis we implemented a Redis-compatible key-value store speaking the Redis Serialization Protocol. Under an identical YCSB2 load it becomes competitive on read throughput beyond roughly 500 000 keys and sustains a growing advantage on range scans, where ordered key storage removes the need for the auxiliary sorted sets Redis requires.

Index Terms—B-Trees, In-Memory Database, Key-Value Store, Memory Layout

I. INTRODUCTION AND MOTIVATION

B⁺-trees are among the most fundamental index structures in database systems, but were long considered inferior for in-memory scenarios. Systems such as Redis rely on hash-based structures with near-constant latency per access. The paper “*B-trees are back: Engineering fast and pageable node layouts*” by Müller, Benson and Leis [1] shows that this is primarily an artifact of the implementation. Through multiple node types and several optimisations such as fingerprinting, hinting, prefix compression and SIMD acceleration, B⁺-trees reach a throughput comparable to the Adaptive Radix Tree (ART) and the Height Optimized Trie (HOT). If this holds, the preference for hash-based indexes in in-memory systems rests on implementation history rather than on a property of the structures, and an ordered index becomes viable where an unordered one is normally assumed. This allows for range queries at no additional cost.

II. ANALYSIS AND SETUP OF THE EXISTING REPOSITORY

The code accompanying the paper is publicly available on GitHub, but was strongly tailored to one specific compiler environment. SIMD intrinsics were embedded directly as AVX2 code, the build system consisted of a hand-written, monolithic Makefile, and dependencies such as TLX had to be placed into the source tree by hand before the project would compile. In this state the code is hard to build outside the authors’ environment.

We therefore restructured the repository into a static library (`libbtree`). The build system was migrated to CMake, which resolves dependencies automatically. The code was updated to compile cleanly under current compilers. The AVX2 intrinsics were moved behind a compiler-independent abstraction that separates the vectorized search from the node layout code. This allows for better portability on platforms which implement accelerated SIMD-intrinsics. In theory, the new codebase can be compiled to architectures other than x86. Overall, these modifications greatly increased reliability and maintainability of the build process.

III. REPRODUCTION OF BENCHMARK RESULTS

The benchmarks of the paper are driven by Python scripts that invoke the benchmarking programs with parameters such as key count, operation count and batch size, store the results as CSV, and visualize them using R. The datasets used were dense and sparse integer keys, Wikipedia article titles and randomly generated URLs. All measurements were taken on a virtual machine hosted in the university datacenter, backed by an AMD EPYC 7702P (64 cores) and provisioned with 4 vCPUs, 64 GB of RAM and a single NUMA node. Each core has 64 KiB of L1 and 512 KiB of L2 cache, alongside 64 MiB of shared L3; the system runs Debian 13 with kernel 7.0.4. The benchmark workload is synchronous and single-threaded and therefore executes on one core, with dedicated cores and memory ruling out contention from co-located tenants. This setup nevertheless differs from the bare-metal machine of the original publication, which must be kept in mind when comparing absolute numbers. We compiled with a newer compiler version, but deliberately kept the optimization and architecture flags from the original Makefile unchanged.

Reproducing the results proved considerably more laborious than expected. The datasets are only described in prose and are not included in the repository, so we first regenerated them from those descriptions. Benchmarks run against these self-generated datasets crashed reproducibly, and the cause could not be determined from the code alone, suggesting that the inputs depend on properties the description does not capture. Not all figures could be unambiguously attributed to a benchmark script either, so the exact configuration behind a published curve sometimes was unclear.

After getting in touch with the authors, we were provided with the original datasets. This removed the crashes and

reduced the discrepancies, but did not eliminate them: our absolute throughput values still deviate from the published ones, in places by orders of magnitude. The qualitative behaviour, however, is preserved. For example, the relative ordering of the data structures and the shape of the curves over growing key counts match, so the papers conclusions are not contradicted by our runs, only their absolute scale is. We cannot attribute the remaining gap to a single cause: the hardware, newer toolchain and possibly different versions of the benchmarking scripts all plausibly contribute and cannot be cleanly separated. Therefore, these deviations are left as unresolved.

IV. IMPLEMENTATION OF A KEY-VALUE STORE BASED ON B-TREES

Based on `libbtree`, a Redis-compatible key-value store was built. The first step was serialization and deserialization of the Redis Serialization Protocol (RESP, Version 2), a text-based encoding featuring strings, integers and arrays. These values are sent over a TCP connection using Boost.Asio; the server parses incoming commands and processes the requests on `libbtree` accordingly. Implemented commands include most basic key-value commands (GET, SET, MGET, MSET, DEL, EXISTS, INCR/DECR and INFO), as well as an experimental adaptation of sorted set operations from redis (ZADD, ZRANGE). Since the protocol is implemented faithfully, the resulting server [2] can be used as a drop-in replacement for Redis.

To compare both systems, the YCSB2 program was adapted to use a Redis client instead of direct B-Tree operations, so that an identical load can be directed at either server over the local network. All measurements use Workload C, consisting of 100 % read accesses drawn from a Zipf distribution. Insertion and retrieval use batched MGET and MSET commands of 10000 operations each to minimize network overhead, and the time for a set number of operations is measured.

Figure 1 shows the throughput of MGET commands with different database sizes. Starting at about 500 000 keys in the database, the system using B-Trees gets competitive. At around 2.6 million keys, Redis’s performance starts benefiting from the theoretical $O(1)$ performance of hash lookups. From here, it stays almost completely flat, while the B-Tree performance is still growing, resembling the expected $O(\log n)$ curve.

A structural advantage of B⁺-Trees is their ordered key storage, which allows range scans without additional data structures. This is not possible in a hash-based structure, which is why Redis provides this functionality through sorted sets.

Figure 2 shows the throughput of ZRANGE commands with different database sizes. With very small key counts, both perform very similar. However, redis has a much steeper increase in duration. At around 3 million keys, redis scan take about twice as long. The steep gradient of redis flattens at this point and increases very similarly to the B-Tree, but remains behind in performance.

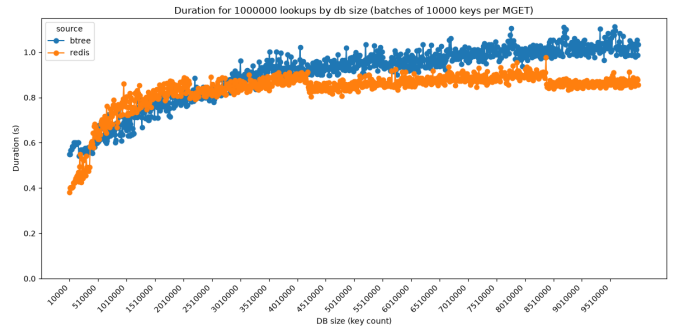


Fig. 1: YCSB2 Workload C: Throughput B-Tree vs. Redis by database size.

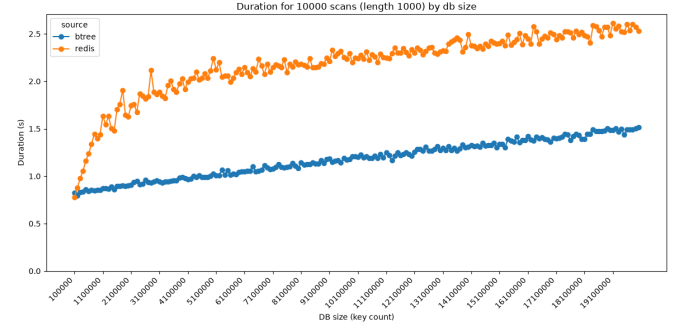


Fig. 2: Throughput of range scans for B-Tree vs. Redis by database size.

V. CONCLUSION

Beyond disk-based databases, the optimized B⁺-trees described in the paper are a solid foundation for competitive in-memory key-value stores. Our Redis-compatible store shows this concretely: it reaches competitive read throughput on sufficiently large datasets, and its ordered key storage gives it a structural advantage on range scans that a hash-based store can only match by maintaining an additional index.

The reproduction was the more instructive part of the project. The published code alone is not sufficient for reproducing the benchmarking results. The datasets, the machine configuration and an unambiguous mapping from figures to scripts were needed as well, and only the first of these could be recovered, by contacting the authors directly. In a field where performance claims are the primary contribution, experimental artifacts deserve the same care in publication as the source code.

REFERENCES

- [1] M. Müller, L. Benson, and V. Leis, “B-trees are back: Engineering fast and pageable node layouts,” *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 1–26, 2025.
- [2] J. Gruhler, M. Bellm, A. Rico Martin, and M. Raba, “btree_redis: Redis-compatible in-memory key-value store based on optimized B⁺-trees,” https://github.com/kantholz44mm/btree_redis, 2026.